

Αναφορά Sign Recognition Project: Αναγνώριση Αμερικανικής Νοηματικής Γλώσσας (ASL)

Simone Pellegatta (A.M.: 1127692), Σοφία Αικατερίνη Μηχανετζή (A.M.: 1100945)

25 Ιουνίου 2026

Περίληψη

Η παρούσα αναφορά περιγράφει την υλοποίηση συστήματος αναγνώρισης της Αμερικανικής Νοηματικής Γλώσσας (ASL) σε πραγματικό χρόνο. Το σύστημα γεφυρώνει το γραφικό περιβάλλον του LabVIEW με υπερσύγχρονα μοντέλα τεχνητής νοημοσύνης (SigLIP) βασισμένα σε Python, επιτρέποντας την οπτική αναγνώριση χειρονομιών και την αλληλεπίδραση με τον χρήστη.

1 Εισαγωγή

Στην παρούσα εργασία αντιμετωπίστηκε το πρόβλημα της μετάφρασης των χειρονομιών του χρήστη από τη νοηματική γλώσσα σε κείμενο. Για την πειραματική αξιολόγηση του συστήματος, κατασκευάστηκε ένα interface στο LabVIEW, μέσα από το οποίο ο χρήστης μπορεί να αλληλεπιδράσει με την κάμερα, να επιλέξει πηγές εικόνας και να παρατηρήσει τις προβλέψεις του αλγορίθμου.

2 Δομή και Λειτουργία του Συστήματος

2.1 Δυνατότητες Πειραματισμού

Ο χρήστης του συστήματος μπορεί να εξετάσει:

1. Την αναγνώριση των γραμμάτων της Αμερικανικής Νοηματικής Γλώσσας σε πραγματικό χρόνο μέσω του κουμπιού Next.Letter.
2. Την εκφώνηση κειμένου που δημιουργείται μέσω του κουμπιού Speak.

Επίσης, υποστηρίζονται λειτουργίες όπως:

- Εναλλαγή μεταξύ ζωντανής κάμερας και στατικών φωτογραφιών (μέσω του διακόπτη Input, που στην περίπτωση στατικής φωτογραφίας διαβάζει το Filename που καταχωρείται από τον χρήστη).
- Συνένωση γραμμάτων ή κενού (μέσω του κουμπιού Space) για τον σχηματισμό λέξεων.
- Εναλλαγή Αναγνώρισης Νοηματικής ανάμεσα σε δεξιόχειρα και αριστερόχειρα (μέσω του διακόπτη Left Hand / Right Hand που εισάγει ως φωτογραφία την αντίστοιχη κατοπτρική της σε περίπτωση αριστερόχειρα) για τον σχηματισμό λέξεων.

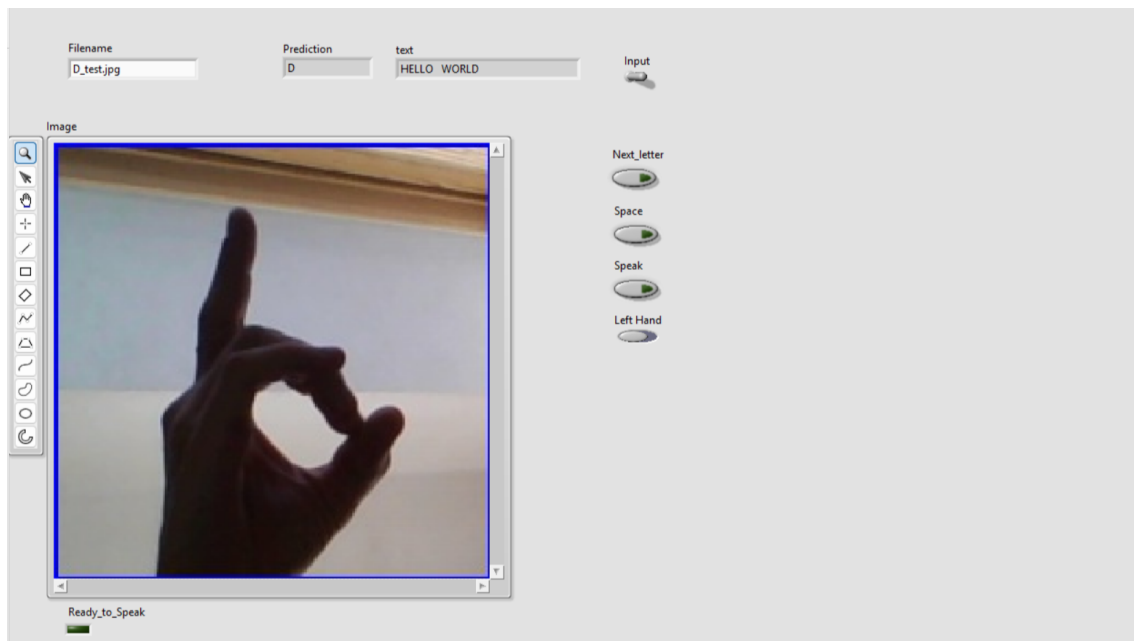
2.2 Γραφικό Περιβάλλον (Front Panel)

Το περιβάλλον χρήστη σχεδιάστηκε στο LabVIEW. Στο Σχήμα 1 φαίνεται το γραφικό περιβάλλον όπου ο χρήστης βλέπει την κάμερα και το αποτέλεσμα της πρόβλεψης, ενώ στο Σχήμα 2 παρουσιάζεται το αντίστοιχο Block Diagram.

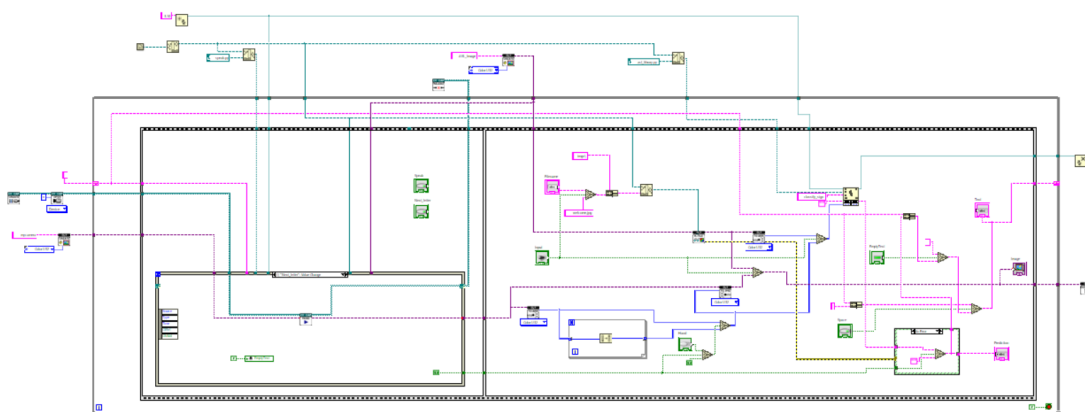
2.3 Αναλυτική περιγραφή λειτουργίας

Το σύστημα λειτουργεί μέσω μιας αρχιτεκτονικής client-server που επιτρέπει την αναγνώριση χειρονομιών της αμερικανικής νοηματικής γλώσσας σε πραγματικό χρόνο.

Αρχικά, ο χρήστης παρέχει δεδομένα εισόδου είτε μέσω ζωντανής ροής κάμερας είτε επιλέγοντας μια στατική εικόνα. Αυτά τα δεδομένα εισόδου καταγράφονται και υποβάλλονται σε επεξεργασία



Σχήμα 1: Το Interface του συστήματος στο LabVIEW, όπου διακρίνονται η λήψη εικόνας και ο δείκτης αποτελέσματος.



Σχήμα 2: Το Block Diagram του συστήματος στο LabVIEW, όπου διακρίνονται η λειτουργία του συστήματος αναγνώρισης της Αμερικάνικης νοηματικής γλώσσας.

από τη γραφική διεπαφή που αναπτύχθηκε στο LabVIEW, η οποία είναι υπεύθυνη για την απόκτηση εικόνας και την αλληλεπίδραση του χρήστη.

Μόλις ληφθεί μια εικόνα, αποστέλλεται μέσω αιτήματος HTTP σε έναν διακομιστή backend που υλοποιείται σε Python. Αυτός ο διακομιστής φιλοξενεί το μοντέλο βαθιάς μάθησης SigLIP, το οποίο επεξεργάζεται την εικόνα και εξάγει σημαντικά χαρακτηριστικά. Το μοντέλο στη συνέχεια υπολογίζει βαθμολογίες πρόβλεψης για κάθε πιθανό γράμμα και επιλέγει το πιο πιθανό.

Η πρόβλεψη επιστρέφεται στον πελάτη LabVIEW μέσω μιας απόκρισης HTTP. Η διεπαφή στη συνέχεια εμφανίζει το αναγνωρισμένο γράμμα σε πραγματικό χρόνο, επιτρέποντας στον χρήστη να σχηματίζει σταδιακά λέξεις χρησιμοποιώντας πρόσθετα στοιχεία ελέγχου όπως Next Letter και Space.

Τέλος, το σύστημα παρέχει προαιρετική λειτουργικότητα μετατροπής κειμένου σε ομιλία, επιτρέποντας τη μετατροπή του παραγόμενου κειμένου σε προφορική έξοδο.

2.4 Εξοπλισμός και Λογισμικό

Η αρχιτεκτονική του συστήματος απαιτεί τον διαχωρισμό του λογισμικού, όπως φαίνεται στον παρακάτω Πίνακα 1.

Ο διαχωρισμός αυτών των επιπέδων και η χρήση διαφορετικών εκδόσεων Python (32-bit και

Επίπεδο (Layer)	Λογισμικό	Αρχιτεκτονική
Γραφικό Περιβάλλον	LabVIEW	32-bit
Ενδιάμεση Γέφυρα	Python (Requests)	32-bit
Διακομιστής AI (Server)	Python (Flask, PyTorch)	64-bit

Πίνακας 1: Η κατανομή του λογισμικού και των εκδόσεων.

64-bit) αποτέλεσε μονόδρομο για την επιτυχή υλοποίηση του συστήματος, για τους εξής τεχνικούς λόγους:

- **Ασυμβατότητα Βιβλιοθηκών (Library Compatibility):** Το γραφικό περιβάλλον και η λήψη εικόνας (NI Vision) του LabVIEW λειτουργούν σε περιβάλλον 32-bit, αναγκάζοντας το Python Node να καλεί αποκλειστικά 32-bit Python. Ωστόσο, τα σύγχρονα και βαριά μοντέλα Deep Learning (όπως το SigLIP) και βιβλιοθήκες όπως το PyTorch, δεν υποστηρίζουν πλέον συστήματα 32-bit και απαιτούν αυστηρά 64-bit περιβάλλον.
- **Όρια Μνήμης (Memory Limits):** Μια εφαρμογή 32-bit έχει αυστηρό όριο στη χρήση μνήμης RAM (περίπου 2-4 GB), το οποίο δεν επαρκεί για τη φόρτωση ενός νευρωνικού δικτύου. Ο διακομιστής 64-bit απελευθερώνει το σύστημα από αυτόν τον περιορισμό.
- **Αποσύνδεση Διεργασιών (Decoupling):** Μέσω της αρχιτεκτονικής Client-Server και της χρήσης HTTP requests, η ροή του βίντεο και η απόκριση του Interface στο LabVIEW παραμένουν ομαλές και ανεπηρέαστες από τον βαρύ φόρτο της επεξεργασίας του AI.

2.5 Μαθηματικό Υπόβαθρο

Η ταξινόμηση της εικόνας γίνεται από το νευρωνικό δίκτυο (SigLIP). Αφού το μοντέλο επεξεργαστεί τον πίνακα pixels, εξάγει ένα διάνυσμα score $z_1, z_2, \dots, z_K$ για κάθε πιθανό γράμμα. Η τελική πιθανότητα p_i για το γράμμα i υπολογίζεται μέσω της συνάρτησης Softmax:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

όπου $K = 26$ είναι ο αριθμός των γραμμάτων του λατινικού αλφαβήτου. Ο διακομιστής επιστρέφει το γράμμα με τη μέγιστη πιθανότητα, με αποτέλεσμα να καταχωρείται στο σύστημα ως η καινούρια πρόβλεψη.

2.6 Συμπεράσματα

Συμπερασματικά, η υλοποίηση του συγκεκριμένου project ξεπέρασε επιτυχώς τους περιορισμούς συμβατότητας των διαφορετικών λογισμικών, δημιουργώντας ένα λειτουργικό και εύχρηστο εργαλείο προσβασιμότητας. Η γέφυρα επικοινωνίας λειτουργήσε άψογα, αποδίδοντας την πρόβλεψη της νοηματικής σε πραγματικό χρόνο. Μελλοντικές βελτιώσεις θα μπορούσαν να περιλαμβάνουν δυναμική αναγνώριση κίνησης, και όχι μόνο στατικών γραμμάτων, καθώς επίσης και την επέκταση σε άλλες νοηματικές γλώσσες εκτός της Αμερικανικής.

Αναφορές

- [1] National Instruments. *LabVIEW and NI Vision Development Module Documentation*. <https://www.ni.com>
- [2] Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*.
- [3] Hugging Face. *Alphabet-Sign-Language-Detection*. <https://huggingface.co/prithivMLmods/Alphabet-Sign-Language-Detection>
- [4] OpenLpVision Reference Documentation. <https://openlpvision.org/docs/>